DragonCandy

CONFIDENTIAL — INTERNAL USE ONLY

Donny AI Cost Architecture & Token Efficiency Strategy

Economic governance for Donny AI across DragonCandy's marketplace platform

Engineering | Product | Leadership

DATE
May 3, 2026

VERSION
1.0 — Approved

STATUS
Ready for Implementation

Executive Summary

Donny AI is DragonCandy's intelligence layer — powering campaign generation, creator matching, social media management, and conversational assistance across every surface of the platform. As Donny's capabilities expand (social media integration, Chrome extension, SMS, embeddable SDK), AI API costs scale with usage. This document establishes the economic governance that keeps Donny viable at every stage of growth — from the current pre-revenue phase through a multi-million dollar ARR business.

> **Core principle:** Use the cheapest model that produces acceptable output for each task, make token budgets invisible to users, and tie AI spend to revenue so the economics improve as the business scales. Every primary user flow under 10 keystrokes by Month 6 — Donny's cost architecture must never create hesitation that works against that north star.

What Donny AI Does

Donny powers five categories of capability across the DragonCandy platform:

- **Campaign generation** — reads restaurant URLs, generates campaign briefs in 60 seconds via `donny-campaign-generate`
- **Creator matching & scoring** — ML-style ranking of creators against campaign criteria via `donny-creator-match`
- **Conversational assistance** — 21-tool agent answering questions, managing campaigns, searching creators via `donny-orchestrator` / `donny-chat`
- **Scheduling & social media** — Auto-Pilot content calendars, caption generation, cross-platform posting via `donny-schedule`
- **Ambient nudges** — proactive notifications about applications, content submissions, and matches via `donny-nudge-frame`

The Problem

Without economic governance, Donny's costs scale linearly with platform adoption — and without model routing, the default is to use expensive frontier models for tasks that cheap models handle equally well. The current implementation uses Sonnet (at ~$3/$15 per million tokens) for scheduling decisions and creator scoring that Haiku (at ~$0.25/$1.25 per million tokens) handles perfectly well. That's a 12x cost difference on tasks representing the majority of Donny's call volume.

The Solution

Three interconnected mechanisms: a **model routing matrix** that assigns the cheapest acceptable model to each task, an **invisible credit system** that degrades gracefully before hitting any wall, and a **revenue cap** with a pre-revenue floor that keeps Donny running while the business finds its footing. Together, these keep AI spend under 15% of revenue at every scale point.

Core Decisions Summary

Decision	Choice	Rationale
Model selection strategy	Task-tier routing matrix (T0–T3)	Cheapest acceptable model per task; most Donny tasks are pattern-matching suited to Haiku
User-facing credit system	Invisible with graceful degradation	Visible credits create friction that works against "less typing" north star
Revenue cap enforcement	15% of revenue, $250/mo floor	Hard dollar floor keeps Donny running pre-revenue; percentage takes over as revenue scales
Vendor strategy	Consolidate generative AI on Claude, keep OpenAI embeddings	One fewer billing relationship; embeddings too cheap to justify migration cost
Fine-tuning path	LoRA on open-source models at 1,000–5,000 campaigns	Data flywheel funds eventual escape from per-token API economics

Key Stakeholder Outcome

≤15%
AI SPEND AS % OF REVENUE

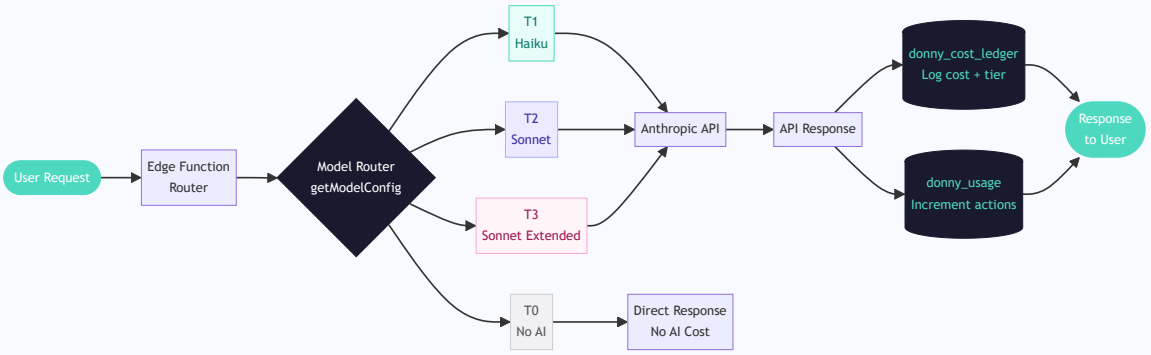
≥60%
TASKS ROUTED TO HAIKU (CHEAPEST TIER)

<5%
USERS HITTING ESSENTIAL MODE MONTHLY

System Architecture Overview

Every Donny AI interaction flows through a deterministic pipeline. The model router is a lookup, not a judgment call — it reads the function name and usage stage, then returns the pre-configured model. Cost is logged synchronously after every API response.

END-TO-END REQUEST FLOW



Developer note: The model router is a pure function — `getModelConfig(functionName, usageStage)` — that returns a config object `{ model, maxTokens, tier }`. It has no side effects and is synchronously testable. All edge functions call this before any API invocation.

1 Model Routing Matrix

Every Donny AI call routes through a tier system. Model selection is a lookup against a static routing table, not a dynamic decision. This eliminates the risk of expensive model calls being made for trivial tasks.

1.1 Tier Definitions

Tier	Model	Cost (Input / Output per MTok)	Max Tokens	When to Use
T0 **No AI**	None	$0	N/A	OAuth flows, analytics fetching, media uploads, scheduled post dispatch (pre-written content), account connections
T1 **Haiku**	claude-haiku-4-5	~$0.25 / $1.25	256–512	Pattern-matching tasks: caption/hashtag generation, reply drafting, nudge framing, scheduling decisions, quick chip generation, platform-specific formatting, simple knowledge base Q&A
T2 **Sonnet**	claude-sonnet-4	~$3 / $15	2048–4096	Multi-step reasoning: campaign wizard conversations, multi-platform cross-posting orchestration, content calendar planning, sponsorship ROI analysis, creator matching/scoring, brand guidelines enforcement
T3 **Sonnet Extended**	claude-sonnet-4	~$3 / $15	8192	Complex multi-tool conversations with full tool use (donny-chat pattern), campaign-from-URL generation with large context windows

> **Cost multiplier context:** T1 Haiku costs approximately 12× less than T2 Sonnet for input tokens and 12× less for output. Routing a task from T2 to T1 on a function called 10,000 times per month represents a meaningful line-item reduction — and most Donny tasks are pattern-matching, not multi-step reasoning.

1.2 Edge Function Migration Table

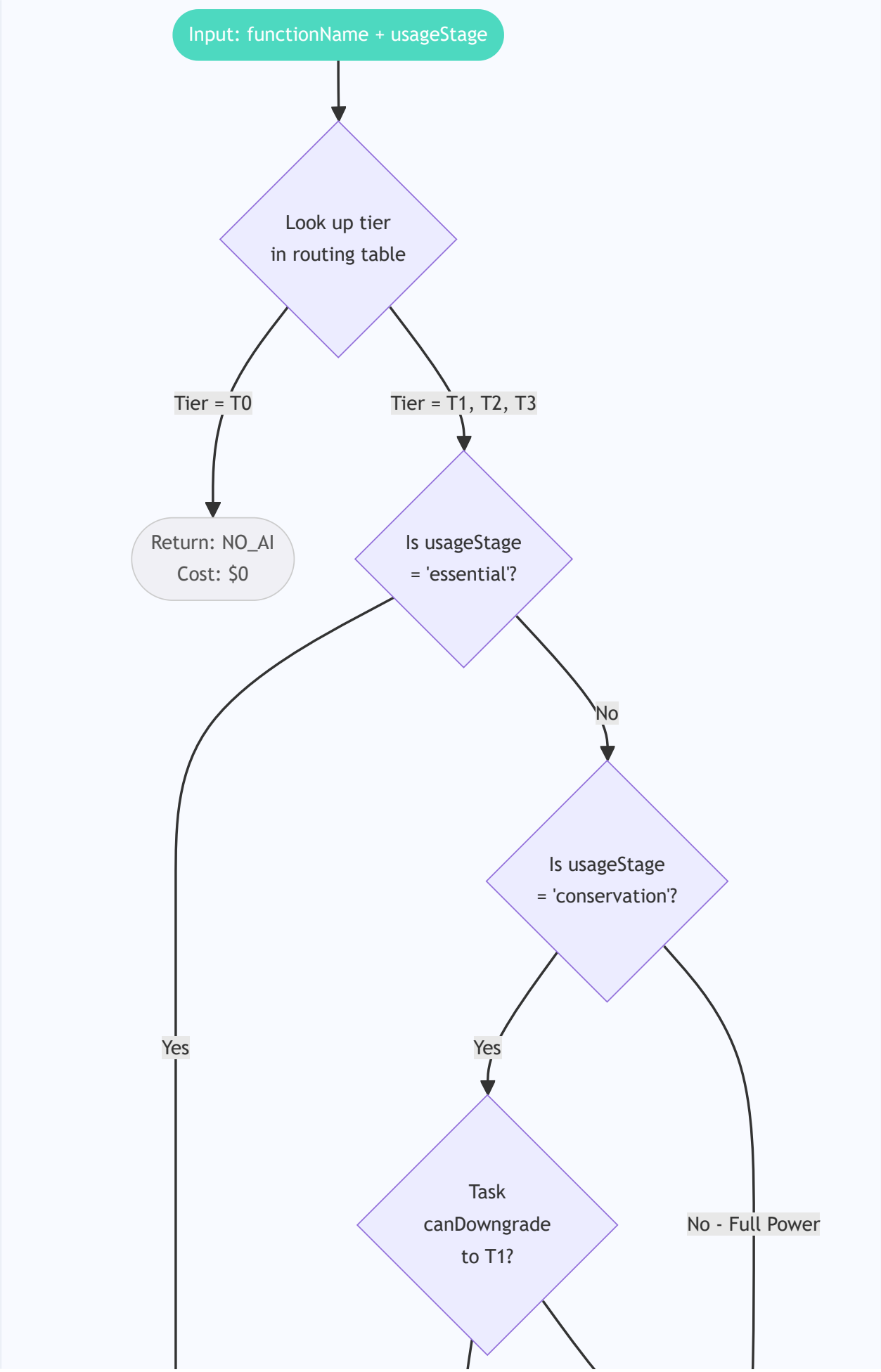
Current production edge functions mapped to their new tier assignments. Functions already at their optimal tier are marked "No change."

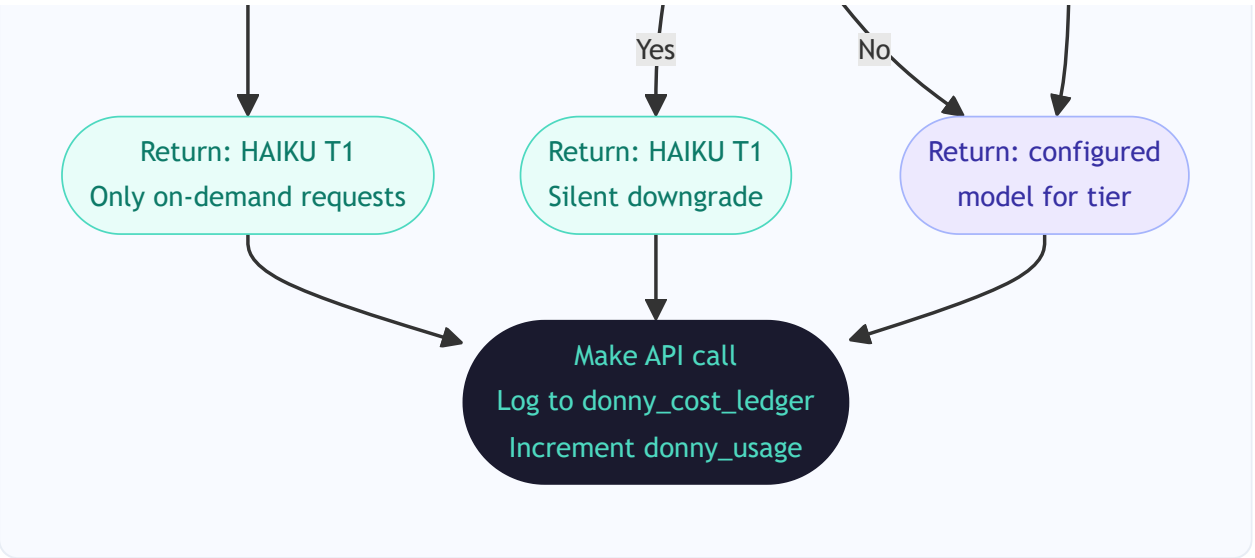
Edge Function	Current Model	Current Max Tokens	New Tier	Action & Rationale
`donny-chat`	Sonnet 4	8192	T3	No change — complex 21-tool conversations require Sonnet Extended
`donny-nudge-frame`	Haiku 4.5	200	T1	No change — already optimally routed
`donny-orchestrator`	Sonnet 4	1024	T2	No change — multi-agent routing with RAG needs Sonnet reasoning
`donny-campaign-preview`	Sonnet 4	4096	T1 / T2 split	**Downgrade:** Mood descriptions and simple storyboards drop to T1. Complex multi-constraint previews stay T2. Route based on input complexity flag.
`donny-schedule`	Sonnet 4	2048	T1	**Downgrade:** Scheduling optimization is pattern matching on time slots and engagement data — no multi-step reasoning required
`donny-campaign-generate`	GPT-4o	N/A	T2	**Migrate:** GPT-4o → Sonnet with tool use for JSON structure. Campaign generation is high-value, justifies Sonnet. Eliminates one API vendor.
`donny-creator-match`	GPT-4o	N/A	T1	**Migrate + Downgrade:** GPT-4o → Haiku. Scoring against criteria is pattern matching — structured

Edge Function	Current Model	Current Max Tokens	New Tier	Action & Rationale
				prompt with JSON tool output is sufficient.
`donny-analytics-alerts`	—	—	T0	Analytics data fetching only — no AI generation required
`donny-dragonshare-score`	—	—	T1	Scoring against engagement criteria — pattern matching at T1
`generate-embedding`	OpenAI text-embedding-3-small	N/A	T0 (OpenAI)	Keep on OpenAI — $0.02/MTok is essentially free, migration cost not justified

1.3 Model Selection Decision Flow

GETMODELCONFIG() LOGIC — DEVELOPER REFERENCE





1.4 Fallback Rule

If a Haiku response fails quality checks (malformed JSON, off-topic output, truncated response), the system retries once at Sonnet and logs the fallback to `donny_cost_ledger` with `fallback: true`.

> **Automatic tier promotion:** If a task category triggers fallback more than 15% of the time over a rolling 7-day window, that category is permanently promoted to the next tier. Tier promotions are reviewed monthly and only kept if the fallback pattern persists. This prevents Haiku degradation from silently inflating costs.

2 Invisible Credit System & Graceful Degradation

2.1 Design Principle

Users never see token counts, model names, tier labels, action balances, or a usage meter. The only visible moment is the Stage 3 upgrade prompt — and even that is framed as Donny talking to the user, not a system wall.

> **Why invisible?** Visible credits create "should I spend a credit on this?" hesitation. That hesitation directly undermines the "less typing = more margin" north star. Salesforce, Adobe, and Intercom all hide usage mechanics for this reason. The meter is an internal accounting tool, not a user experience element.

2.2 The Unit: Donny Actions

Every AI call has a token cost internally. Externally, the abstraction is **Donny actions** — a normalized unit that hides the variance between a cheap Haiku caption (~800 tokens, ≈$0.0001) and an expensive Sonnet multi-tool conversation turn (~6,000 tokens, ≈$0.10).

Task Tier	Actions Cost	Typical Token Count	Examples
T0 No AI	0 actions	0	Scheduled post dispatch, OAuth, analytics fetch
T1 Haiku	1 action	~800 tokens	Caption generation, hashtag suggestions, nudge framing, scheduling
T2 Sonnet	3 actions	~3,000 tokens	Creator matching, campaign calendar planning, ROI analysis
T3 Sonnet Extended	5 actions	~6,000–10,000 tokens	Full Donny chat with tool use, campaign-from-URL generation
Auto-Pilot daily cycle	10 actions flat	~12,000 tokens	Full weekly content plan generation and scheduling

2.3 Tier Allocations (Monthly)

Subscription Tier	Monthly Donny Actions	Approx Token Budget	Auto-Pilot Eligible	DragonDash Rush
Free ($0)	50	~500K tokens	No	No
Starter ($149)	500	~5M tokens	No	Yes (+ surcharge)
Growth ($499)	2,000	~20M tokens	Yes (standard frequency)	Yes (+ surcharge)
Pro ($999)	10,000	~100M tokens	Yes (high frequency)	Yes (included)
Enterprise (custom)	Custom	Custom	Yes (custom schedule)	Yes (included)

2.4 Graceful Degradation — Three Stages

STAGE 1

Full Power

0–80%

- All features at designed tier
- Auto-Pilot at full frequency
- Proactive nudges active
- No user notification
- T2/T3 for complex tasks

STAGE 2

Conservation Mode

80–100%

- T2 tasks silently shift to T1
- Auto-Pilot frequency halved
- Proactive suggestions reduce
- No user notification
- On-demand still fully served

STAGE 3

Essential Mode

100%+

- On-demand only, at T1 Haiku
- Auto-Pilot paused
- Proactive nudges stopped

- Single non-blocking message
- Overage at $0.10–0.25/action

2.5 Degradation State Machine

USAGE STAGE TRANSITIONS

2.6 User Experience at Each Stage

The goal is that Stage 1 and Stage 2 feel identical to the user. Stage 3 is the only moment a user becomes aware of any limitation — and it is framed as Donny speaking, not a system alert.

Feature	Stage 1: Full Power	Stage 2: Conservation	Stage 3: Essential
Campaign brief generation	Sonnet (T2) — rich detail	Haiku (T1) — slightly shorter	Haiku (T1) — works
Caption suggestions	Available instantly	Available (Haiku)	Available (Haiku)
Creator matching	Haiku scoring (T1)	Haiku scoring (T1)	Haiku scoring (T1)
Auto-Pilot posting	Full weekly schedule	Every other day	Paused
Proactive nudges	Full frequency	Reduced	Stopped
What user sees	Nothing different	Nothing different	Donny: "I've been working hard this month. Upgrade to keep me at full speed, or add credits."

3 Revenue Cap Governance

3.1 The Rule

> **AI API spend is hard-capped at 15% of monthly revenue, with a $250/month pre-revenue floor.** This mirrors the kill-switch discipline from PROJECT_CONTEXT: AI spend is treated like headcount — it grows with revenue, not ahead of it.

3.2 Pre-Revenue Floor Logic

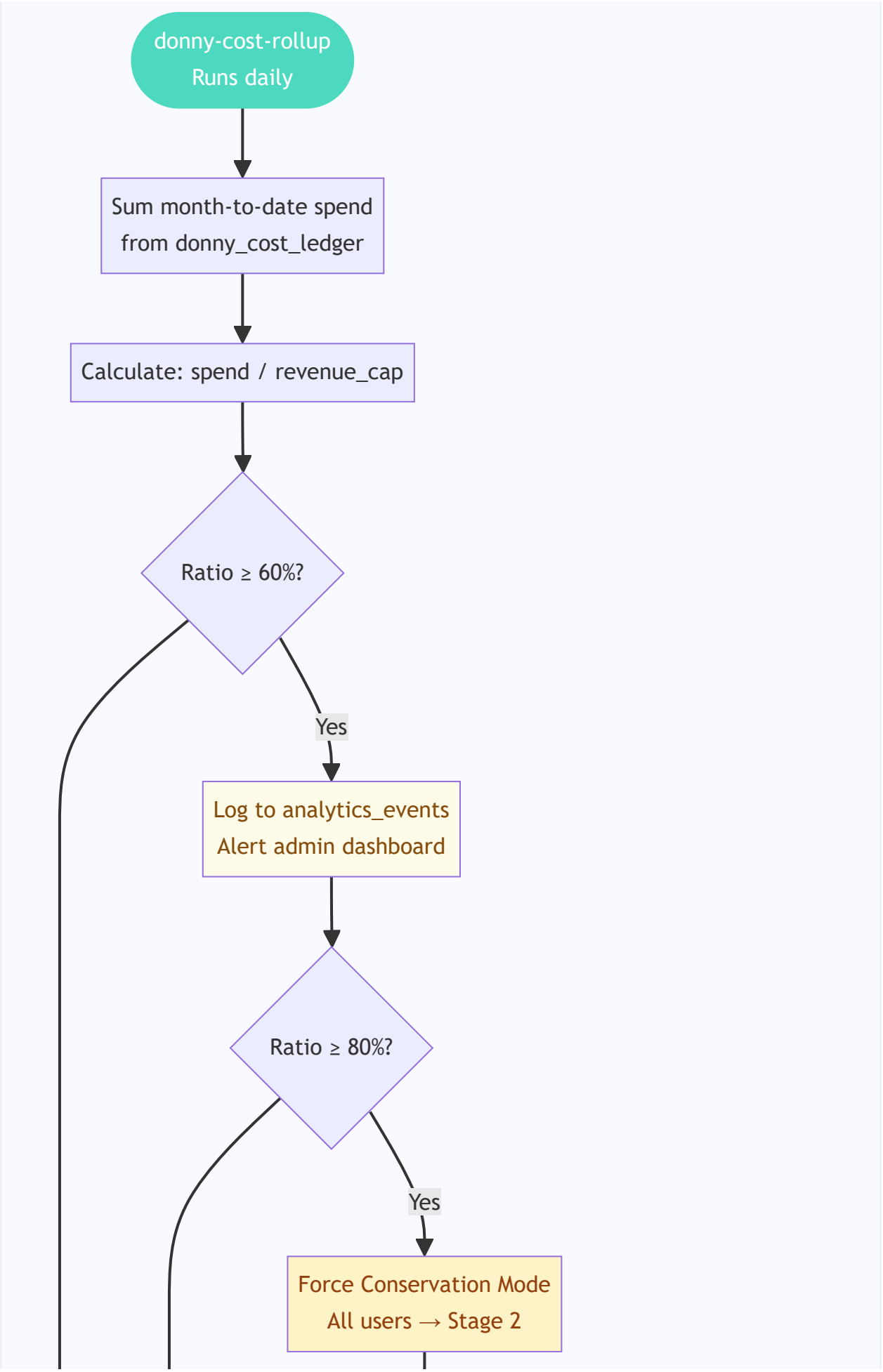
At $0 revenue, 15% = $0, which is not functional. The **$250/month floor** holds until monthly revenue exceeds **$1,667** — at which point 15% × $1,667 = $250 and the percentage cap takes over naturally. At Haiku-dominant routing, $250 buys approximately 50–100 million tokens per month — far more than the current ~30 organic users could consume.

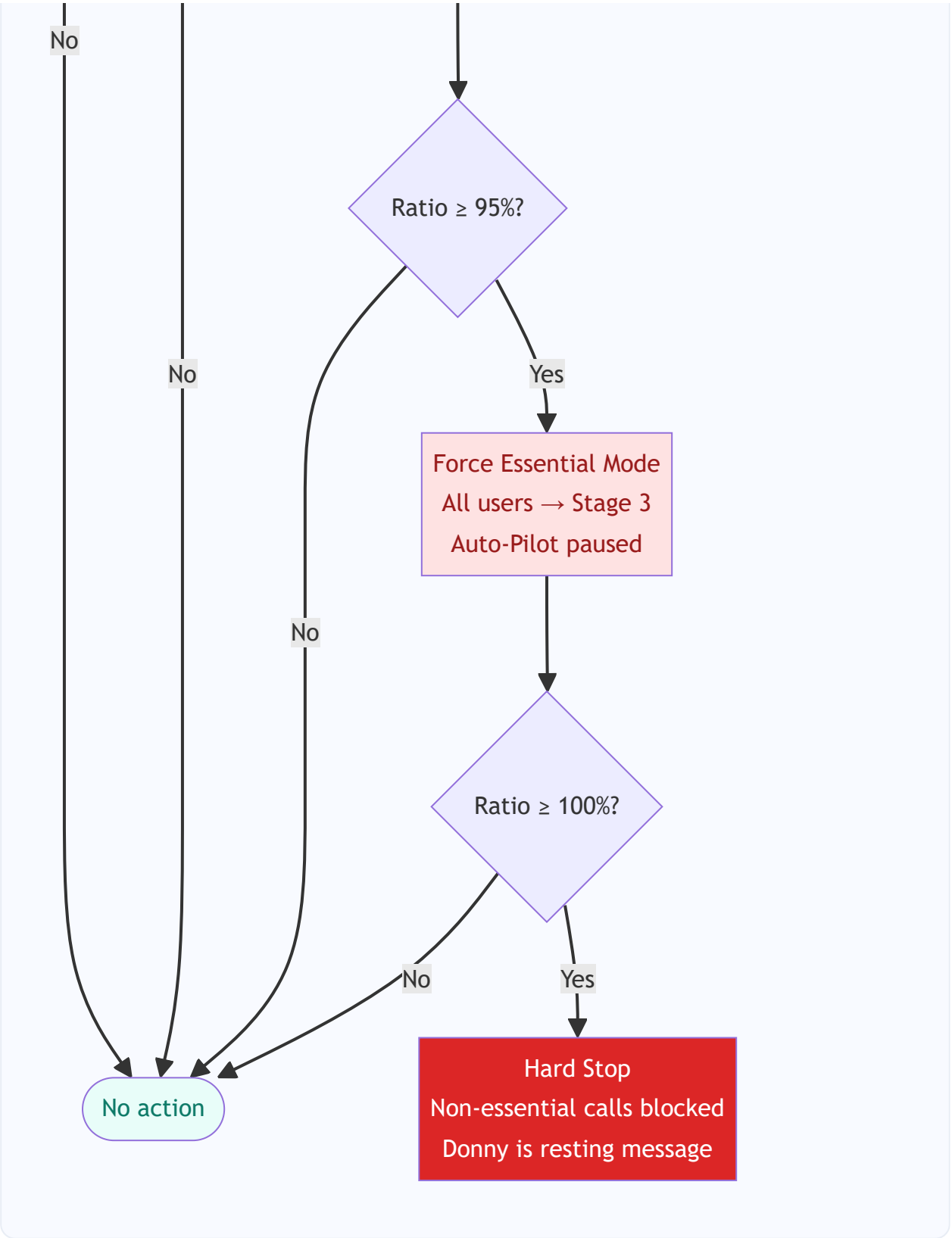
3.3 Alert Threshold Table

Threshold	Action	Who Is Affected
60% of cap	Internal alert logged to `analytics_events`, visible in admin dashboard	Engineering/ops only
80% of cap	Conservation mode forced platform-wide — all users shift to Stage 2 regardless of individual allocation	All users (silently)
95% of cap	Essential mode forced platform-wide — only on-demand T1 requests served; Auto-Pilot and proactive features paused	All users (Donny less chatty)
100% of cap	Hard stop on non-essential AI calls. Only mission-critical paths continue (campaign creation wizard, payment-related Donny actions). Everything else returns graceful "Donny is resting" message.	All users (visible, friendly)

3.4 Alert Escalation Flow

DAILY ROLLUP → ALERT ESCALATION LOGIC



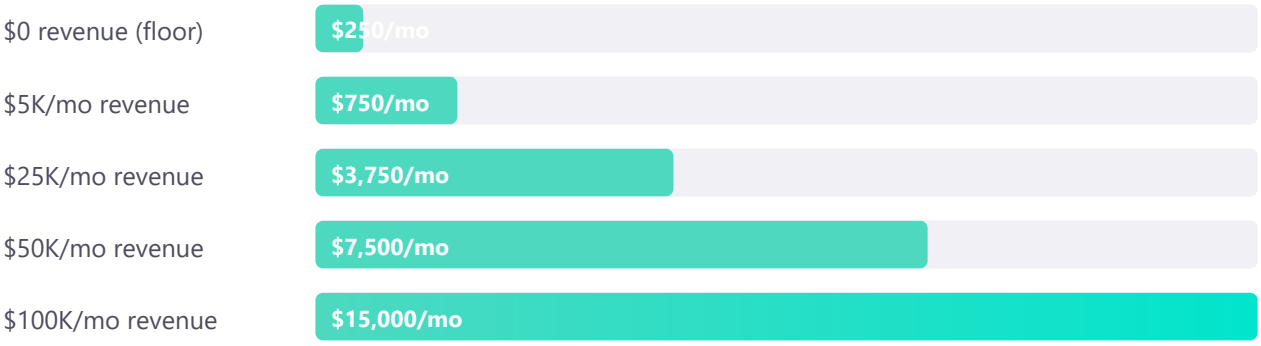


3.5 Cap Scaling with Revenue

Monthly Revenue	AI Spend Cap (15%)	Per-User Budget (100 users)	Per-User Budget (1,000 users)
$0 (pre-revenue)	**$250 floor**	$2.50	$0.25
$5,000	$750	$7.50	$0.75
$25,000	$3,750	$37.50	$3.75
$50,000	$7,500	$75.00	$7.50
$100,000	$15,000	$150.00	$15.00

> **Growth insight:** As revenue grows, the per-user budget grows proportionally — enabling promotion of more tasks from Haiku to Sonnet, increased Auto-Pilot frequency, and premium features unlocked at lower tiers. The model routing matrix is designed to evolve with the revenue curve.

3.6 Revenue-to-AI-Budget Visualization



3.7 Kill-Switch Integration

If AI spend per user exceeds the revenue that user generates (LTV:AI-cost ratio < 2:1 at the individual level), that signals either tightening their tier allocation or surfacing an upgrade prompt. This mirrors the portfolio-level kill-switches from PROJECT_CONTEXT (LTV:CAC < 2:1). The `donny_cost_ledger` table provides the data for this analysis per-user and per-tier.

4 Vendor Consolidation

4.1 Short-Term (Pre-Launch, 2–4 Weeks): Migrate GPT-4o to Claude

Two edge functions currently use OpenAI's GPT-4o. Migrating both to Anthropic's Claude eliminates one billing relationship for generative AI tasks, enables consistent model routing governance, and simplifies secrets management to a single Anthropic API key (plus OpenAI for embeddings only).

Edge Function	Current	Target	Rationale
`donny-campaign-generate`	GPT-4o	Sonnet (T2)	Structured JSON via tool use. Campaign generation is high-value output, justifies Sonnet.
`donny-creator-match`	GPT-4o	Haiku (T1)	Scoring against criteria is pattern matching — well-structured prompt with JSON tool output handles it.

> **Result:** One API key (Anthropic) for all generative AI. One API key (OpenAI) for embeddings only. Simpler secrets management, one billing dashboard, and consistent model governance.

4.2 Medium-Term (Post-Launch, 3–6 Months): Keep OpenAI Embeddings

`text-embedding-3-small` at $0.02/MTok is essentially free. Re-embedding all knowledge chunks in `donny_knowledge` to switch embedding providers costs engineering time with no meaningful savings. The `donny-orchestrator/rag.ts` module has a clean abstraction — the embedding call is isolated, so swapping later is a single-function change if Anthropic releases a standalone embedding model.

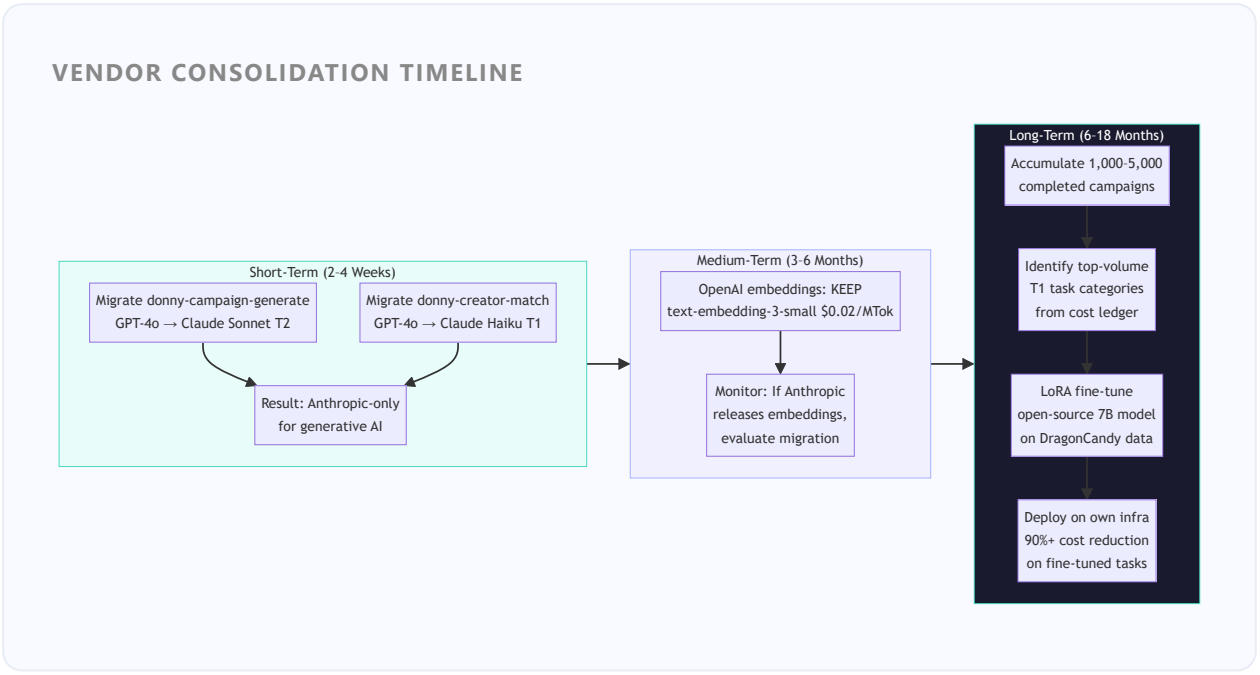
4.3 Long-Term (1,000–5,000 Campaigns): Fine-Tuned Open-Source Models

The cost architecture creates the data to make fine-tuning evidence-based rather than speculative:

- `donny_cost_ledger` tracks which task categories consume the most tokens
- Model routing fallback logs identify which Haiku tasks frequently need Sonnet rescue
- Volume by task category shows where LoRA fine-tuning delivers the highest ROI

When the data is sufficient, the highest-volume T1 tasks — caption writing, hashtag generation, reply drafting, scheduling decisions — are prime candidates for LoRA fine-tuning on a small open-source model (7B parameters). These tasks are repetitive, pattern-matchable, and domain-specific. A fine-tuned model on own infrastructure could reduce per-call cost by 90%+ for these tasks.

4.4 Consolidation Roadmap



5 Data Architecture

Two new tables, following the ledger-first architecture principle already embedded in the codebase. Schema and RLS are defined before any routing code. Every AI call is logged synchronously before returning a response.

5.1 `donny_cost_ledger` — Per-Call Cost Tracking

Column	Type	Purpose
`id`	uuid (PK)	Primary key — gen_random_uuid()
`user_id`	uuid (FK → profiles)	Which user triggered the call
`edge_function`	text	Which edge function made the call (e.g., "donny-orchestrator")
`model`	text	Model used: "claude-haiku-4-5", "claude-sonnet-4", or "none"
`tier`	text	T0 / T1 / T2 / T3
`input_tokens`	integer	Tokens in the request (from Anthropic usage object)
`output_tokens`	integer	Tokens in the response (from Anthropic usage object)
`estimated_cost_usd`	numeric(10,6)	Calculated: (input_tokens × input_rate) + (output_tokens × output_rate)
`fallback`	boolean	True if this was a tier-promotion retry (Haiku → Sonnet fallback)
`created_at`	timestamptz	Call timestamp — default now()

RLS policy: Admin-only read access. Edge functions write via service role. No user-level read access (users never see their cost data directly).

Indexes: `(user_id, created_at DESC)`, `(edge_function, tier, created_at)`, `(fallback, created_at)` for fallback rate analysis.

5.2 `donny_usage` — Per-User Action Tracking

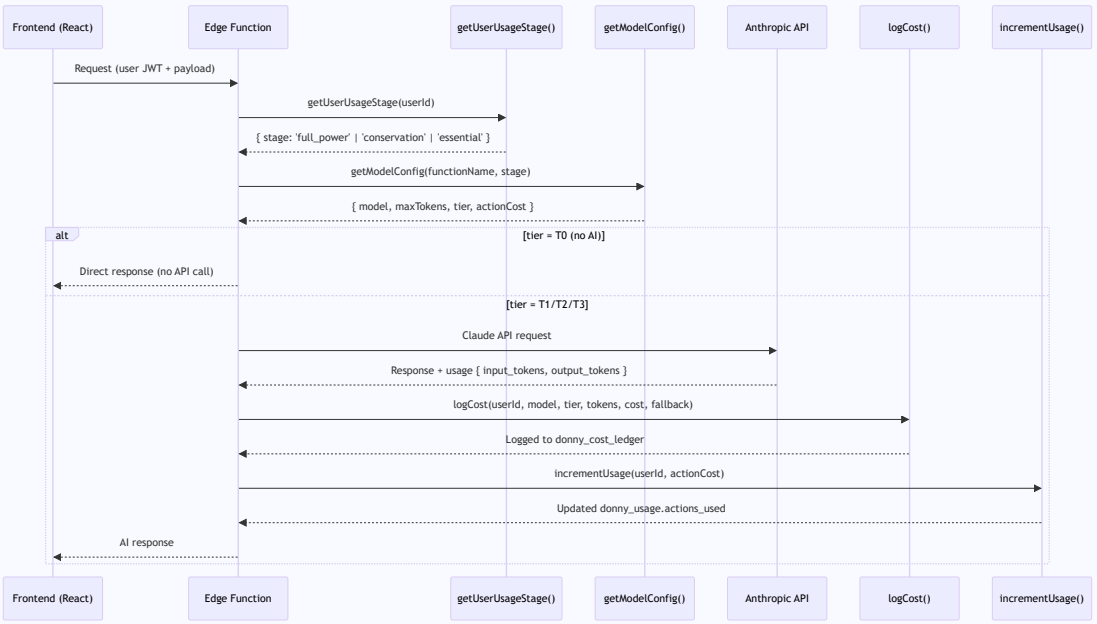
Column	Type	Purpose
`id`	uuid (PK)	Primary key
`user_id`	uuid (FK → profiles)	One row per user per billing period
`period_start`	date	First day of billing period (resets monthly)
`actions_used`	integer	Running total of Donny actions consumed this period
`actions_budget`	integer	Monthly allocation for user's current subscription tier
`current_stage`	text	full_power / conservation / essential — drives model routing decisions
`updated_at`	timestamptz	Last update timestamp

RLS policy: Users can read their own row (but only `current_stage` is ever surfaced in UI). Edge functions update via service role using atomic `actions_used = actions_used + N` to avoid race conditions.

Unique constraint: `(user_id, period_start)` — one row per user per period.

5.3 Data Flow Diagram

API CALL → COST LOG → USAGE UPDATE



6 Social Media Integration Alignment

The Outstand social media integration inherits this cost architecture. The following definitions align social media features with the economic governance defined in this spec. All social media AI calls flow through the same model routing matrix.

6.1 DragonDash Rush Posting Differentiation

Rush multi-platform posting is always a DragonDash event — the most expensive AI action generates the most revenue. When a creator's deliverable is approved, the business sees:

Option	Tier	Action Cost	Revenue Generated	AI Work
"Post now to all platforms"	DragonDash rush	3 actions (T2)	$25–50 surcharge	Simultaneous multi-platform posting with Sonnet-written platform-specific captions, optimized hashtags, cross-tagging
"Schedule for optimal times"	Standard	1 action (T1)	No surcharge	Haiku picks best time per platform, queues posts
"Post to one platform now"	Standard	1 action (T1)	No surcharge	Single-platform post with Haiku caption
"Edit first" / "Skip"	Free	0 actions (T0)	No surcharge	No AI — user writes manually

6.2 Model Routing Per Social Media Feature

6 of 15 social media capabilities are T1 (Haiku), 6 are T2 (Sonnet), and 3 are T0 (no AI). Versus naively assigning Sonnet to every social feature — this routing cuts social media AI cost roughly in half.

Social Media Feature	Tier	Rationale
Caption / hashtag generation	T1 Haiku	Pattern matching — templates + trending data
Engagement hub reply drafting	T1 Haiku	Short-form responses from templates
Content calendar slot suggestions	T1 Haiku	Time-slot optimization from engagement data
UGC detection & reshare prompts	T1 Haiku	Tag/mention detection + templated prompt
Google Business sync posts	T1 Haiku	Format adaptation for GBP — pattern matching
Cross-post caption rewriting	T1 Haiku	Voice/tone adaptation — pattern matching
Growth insights & recommendations	T2 Sonnet	Multi-signal analysis across platforms
Multi-platform simultaneous posting	T2 Sonnet	Complex orchestration across platform APIs
Brand guidelines enforcement	T2 Sonnet	Content review against multiple constraints
Sponsorship ROI report generation	T2 Sonnet	Multi-party analytics synthesis
Auto-Pilot weekly planner	T2 Sonnet	Multi-day content strategy generation
Sponsorship intelligence	T2 Sonnet	Cross-campaign pattern analysis
Scheduled post dispatch	T0 No AI	Pre-written content, just an API call

Social Media Feature	Tier	Rationale
Analytics fetching from Outstand	T0 No AI	Data retrieval only, no generation
Account connection / OAuth	T0 No AI	Authentication flow, no AI involved

6.3 Automation Levels Tied to Subscription Tiers

Automation Level	Available On	Action Cost	Behavior
Manual	All tiers (Free+)	0 actions	User writes captions, picks times, publishes through UI. Donny helps only when explicitly asked.
Assisted (default)	Starter+ ($149)	1–3 actions per post	Donny drafts captions and suggests optimal times. User reviews before publish. Haiku-powered.
Auto-Pilot	Growth+ ($499)	10 actions/day flat	Donny generates weekly content plan, schedules and publishes autonomously, sends daily summary digest.

> **Free-tier social access:** Free users still get social media features — manual mode through the UI with occasional Donny help up to their 50-action monthly limit. This keeps the integration valuable at every tier while reserving expensive automation for paying customers.

7 Success Metrics

All metrics are derived from the `donny_cost_ledger` and `donny_usage` tables. The `donny-cost-rollup` edge function runs daily and surfaces these to an admin dashboard.

Metric	Target	Measurement	Business Signal
AI spend as % of revenue	≤ 15% (or ≤ $250/mo pre-revenue)	`donny_cost_ledger` monthly rollup vs. Stripe revenue	Primary health metric — above 15% triggers review
Haiku task ratio	≥ 60% of all Donny actions at T1	`donny_cost_ledger` tier distribution query	Routing efficiency — lower means Sonnet overuse
Fallback rate per category	< 15% per task category per 7 days	`donny_cost_ledger` fallback flag, grouped by edge_function	Quality signal — above 15% triggers permanent tier promotion
Stage 3 triggers	< 5% of active users per month	`donny_usage` where current_stage = 'essential'	UX health — above 5% means allocations are too tight
Upgrade conversion from Stage 3	> 20% upgrade within 7 days of hitting Stage 3	Subscription events correlated with Stage 3 timestamps	Monetization effectiveness of degradation design
GPT-4o elimination	0 GPT-4o calls within 4 weeks of migration	`donny_cost_ledger` model distribution	Vendor consolidation completion
Per-user AI cost trend	Decreasing month-over-month as Haiku routing expands	`donny_cost_ledger` per-user aggregation, month-over-month delta	Efficiency improvement — should trend down as routing matures
DragonDash AI cost vs. surcharge revenue	Surcharge revenue ≥ 5× AI cost of rush requests	DragonDash orders correlated with T2 cost in ledger	Confirms high-cost AI actions generate matching revenue

Key Stakeholder KPIs

≤15%

AI COST / REVENUE CAP

$250/mo floor pre-revenue

≥60%

HAIKU (T1) TASK RATIO

Routing efficiency target

<15%

FALLBACK RATE PER CATEGORY

Quality vs. cost balance

<5%

USERS HITTING ESSENTIAL MODE

UX health signal

>20%

STAGE 3 → UPGRADE CONVERSION

Monetization effectiveness

0

GPT-4O CALLS POST-MIGRATION

Vendor consolidation target

8 Guiding Principles

CHEAPEST ACCEPTABLE MODEL

Default to Haiku. Promote to Sonnet only when task complexity genuinely demands it. Never use Sonnet because it feels "safer." The burden of proof is on the expensive model.

INVISIBLE ECONOMICS

Users experience Donny's intelligence, not his cost. No meters, no credit counts, no model names surfaced in UI. The only visible moment is Stage 3 — framed as Donny speaking, not a system alert.

GRACEFUL OVER ABRUPT

Degrade quality gradually rather than hitting a wall. Conservation mode before essential mode before hard stop. The user should experience a slightly quieter Donny, not a broken one.

LEDGER-FIRST

Every AI call is logged with cost before returning a response. Follows the payment_ledger discipline already in the codebase. No governance decision is made without data to support it.

DRAGONDASH CAPTURES PREMIUM

The most expensive AI actions (rush multi-platform posting, complex orchestration) generate the most revenue via DragonDash surcharges. High-cost features must pay for themselves — and then some.

REVENUE-ADAPTIVE

The model routing matrix, tier allocations, and feature availability evolve as revenue grows. What's T1/Haiku today might be T2/Sonnet at $50K MRR. The architecture is designed for this evolution.

VENDOR-LIGHT

Minimize API vendor dependencies. One vendor for generative AI, one for embeddings. Fine-tuning on open-source models is the long-term exit from per-token economics entirely.

Appendix: Edge Function Integration Pattern

Every edge function that makes an AI call follows this standard pattern. This is a pseudocode reference — the actual TypeScript implementation lives in each function's `index.ts`. The key invariant: **cost is always logged, usage is always incremented, and model selection is always via the routing table, never hardcoded.**

Standard Donny AI Edge Function Pattern

```
// 1. Auth — always first
const user = await authenticateRequest(req);
if (!user) return unauthorizedResponse();

// 2. Usage stage — read before any AI call
const usageStage = await getUserUsageStage(user.id);
// Returns: 'full_power' | 'conservation' | 'essential'

// 3. Model config — lookup, not judgment
const modelConfig = getModelConfig(functionName, usageStage);
// Returns: { model, maxTokens, tier, actionCost, canMakeCall }
// If tier = T0, returns NO_AI config and caller skips API call

if (modelConfig.tier === 'T0') {
  // No AI call needed — execute directly
  return directResponse(payload);
}

// 4. Build the prompt (function-specific logic here)
const messages = buildMessages(payload, context);

// 5. Call the API (Anthropic only, for T1/T2/T3)
const response = await anthropic.messages.create({
  model: modelConfig.model,
  max_tokens: modelConfig.maxTokens,
  messages,
});

// 6. Quality check — for T1 tasks that might need T2 fallback
if (modelConfig.tier === 'T1' && !isValidResponse(response)) {
  const fallbackConfig = getModelConfig(functionName, usageStage, { force
```

```
  const fallbackResponse = await anthropic.messages.create({ ... });
  await logCost({ ...fallbackConfig, fallback: true, usage: fallbackRespo
  await incrementUsage(user.id, fallbackConfig.actionCost);
  return formatResponse(fallbackResponse);
}

// 7. Log cost — synchronous, before returning
await logCost({
  userId: user.id,
  edgeFunction: functionName,
  model: modelConfig.model,
  tier: modelConfig.tier,
  inputTokens: response.usage.input_tokens,
  outputTokens: response.usage.output_tokens,
  estimatedCostUsd: calculateCost(modelConfig, response.usage),
  fallback: false,
});

// 8. Increment usage — synchronous, before returning
await incrementUsage(user.id, modelConfig.actionCost);

// 9. Return the response
return formatResponse(response);
```

Key Function Signatures

Function	Signature	Location
`getModelConfig`	`(functionName: string, stage: UsageStage, opts?: { forceTier?: Tier }) → ModelConfig`	`supabase/functions/_shared/modelRouter.ts`
`getUserUsageStage`	`(userId: string) → Promise<'full_power' | 'conservation' | 'essential'>`	`supabase/functions/_shared/usageStage.ts`
`logCost`	`(params: CostLogParams) → Promise<void>`	`supabase/functions/_shared/costLogger.ts`
`incrementUsage`	`(userId: string, actionCost: number) → Promise<void>`	`supabase/functions/_shared/usageTracker.ts`
`calculateCost`	`(config: ModelConfig, usage: AnthropicUsage) → number`	`supabase/functions/_shared/costCalculator.ts`

> **Shared utilities location:** All routing, logging, and usage utilities live in `supabase/functions/_shared/` — the existing shared module already used by `donny-chat` for CORS headers and Supabase client initialization. New utilities extend this module without altering existing patterns.

Migration Checklist for Existing Functions

1. Add `getUserUsageStage(user.id)` call immediately after auth
2. Replace hardcoded model string with `getModelConfig(functionName, stage).model`
3. Replace hardcoded `max_tokens` with `modelConfig.maxTokens`
4. Add `logCost()` call after API response, before return
5. Add `incrementUsage()` call after `logCost()`

6. Add quality check + fallback retry for T1 functions that output structured data
7. Test with `npx tsc --noEmit` then `npm run build`
8. Verify new rows appear in `donny_cost_ledger` and `donny_usage` after test call

DragonCandy — Confidential — Internal Use Only
Donny AI Cost Architecture v1.0 — May 3, 2026
Prepared by DragonCandy Engineering & Product